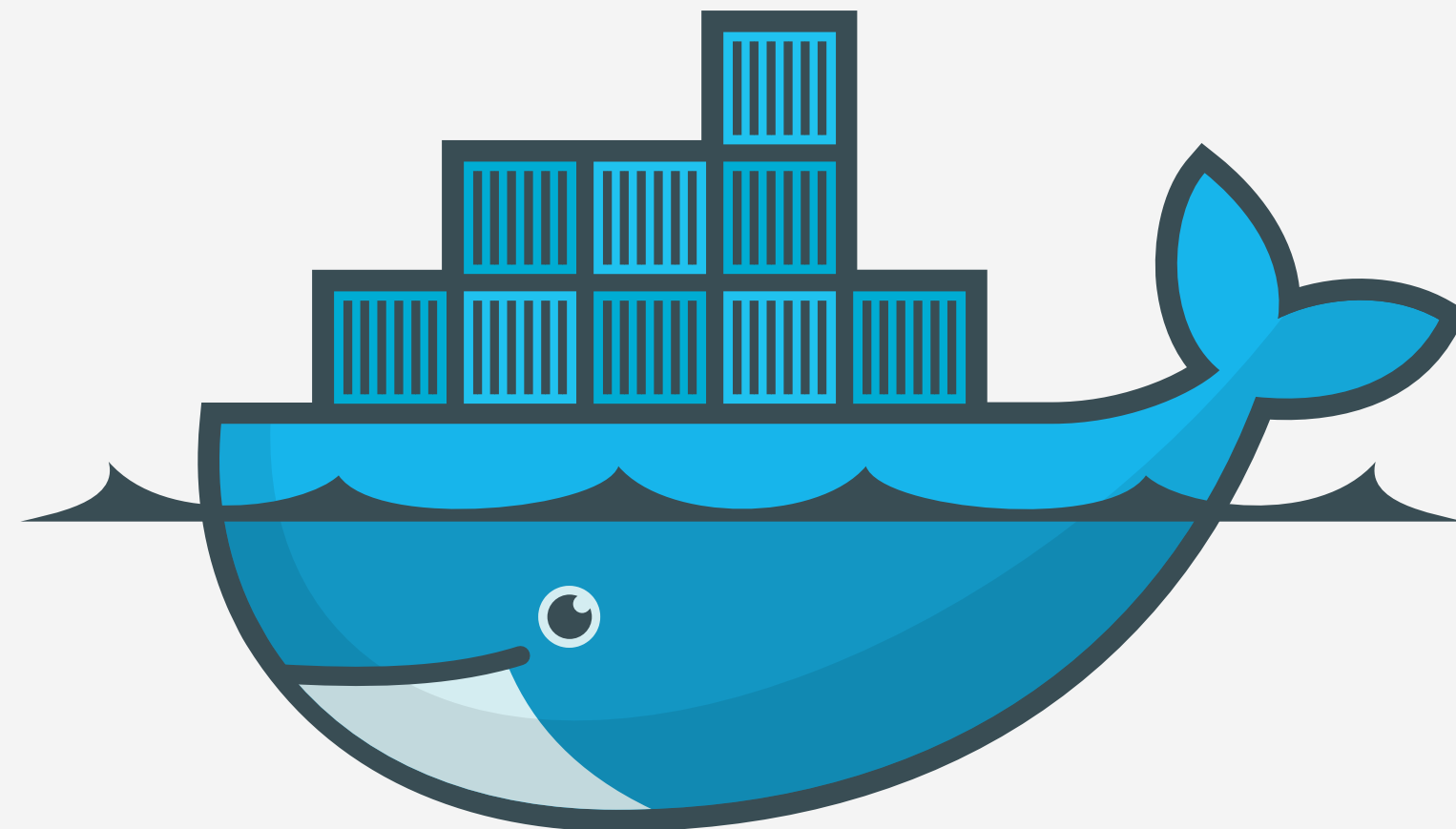




Факултет по Математика и Информатика
ПУ "Паисий Хилендарски"

DOCKER COMPOSE

Витали Атиас



Какво е Docker Compose?

Docker Compose е инструмент за дефиниране и стартиране на multi-container Docker приложения.

Какво е Docker Compose?

Docker Compose е инструмент за дефиниране и стартиране на multi-container Docker приложения.

Ключови характеристики:

- Декларативен подход - дефинирате желаното състояние в YAML файл
- Едно-командно управление - стартираме цялото приложение с една команда
- Reproducibility - лесно споделяне и възпроизвеждане на среди
- Development-friendly - идеален за локална разработка и тестване

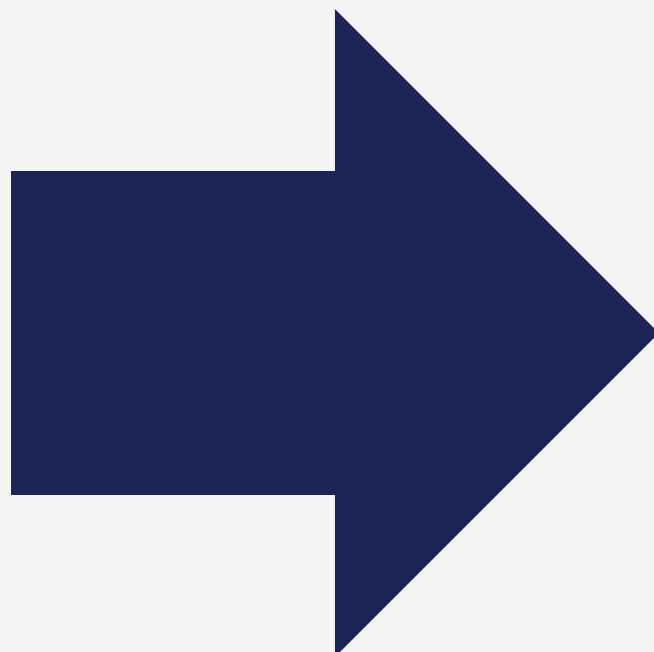
Какво е Docker Compose?

```
# docker-compose.yml
services:
  web:
    image: nginx
  database:
    image: postgres
```

```
# CLI
docker compose up
```

Проблемът без Docker Compose

- Frontend контейнер
- Backend API контейнер
- База данни
- Redis cache
- Nginx reverse proxy



```
# Създаване на network
docker network create myapp-network

# Стартиране на база данни
docker run -d --name db --network myapp-network \
  -e POSTGRES_PASSWORD=secret \
  -v db-data:/var/lib/postgresql/data \
  postgres:15

# Стартиране на Redis
docker run -d --name redis --network myapp-
network redis:7

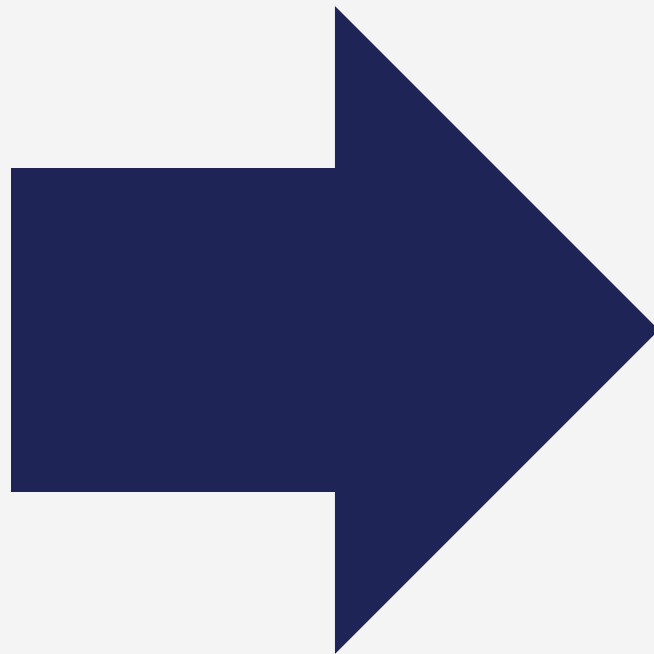
# Стартиране на backend
docker run -d --name api --network myapp-network \
  -e DATABASE_URL=postgresql://db:5432/myapp \
  -e REDIS_URL=redis://redis:6379 \
  -p 5000:5000 \
  myapp-backend:latest

# Стартиране на frontend
docker run -d --name frontend --network myapp-
network \
  -e API_URL=http://api:5000 \
  -p 3000:80 \
  myapp-frontend:latest

# И още много команди...
```


Docker Compose

- Frontend контейнер
- Backend API контейнер
- База данни
- Redis cache
- Nginx reverse proxy



```
# docker-compose.yml
services:
  db:
    image: postgres:15
    environment:
      POSTGRES_PASSWORD: secret
    volumes:
      - db-data:/var/lib/postgresql/data

  redis:
    image: redis:7

  api:
    build: ./backend
    environment:
      DATABASE_URL: postgresql://db:5432/myapp
      REDIS_URL: redis://redis:6379
    ports:
      - "5000:5000"
    depends_on:
      - db
      - redis

  frontend:
    build: ./frontend
    environment:
      API_URL: http://api:5000
    ports:
      - "3000:80"
    depends_on:
      - api

volumes:
  db-data:

# CLI
docker compose up -d
```

ОСНОВНИ КОНЦЕПЦИИ

Services (Услуги)

Service е основната единица в Docker Compose. Всеки service представлява един или повече контейнери, които изпълняват определена функция във вашето приложение.

```
services:
  web:
    # Image & Build
    image: nginx:latest
    build: ./path

    # Ports
    ports:
      - "80:80"

    # Environment
    environment:
      KEY: value
    env_file:
      - .env

    # Volumes
    volumes:
      - ./src:/app
      - data:/var/lib/data

    # Networks
    networks:
      - frontend

    # Dependencies
    depends_on:
      - db

    # Health Check
    healthcheck:
      test: ["CMD", "curl", "-f", "http://localhost/health"]
      interval: 30s
      timeout: 10s
      retries: 3

    # Restart
    restart: unless-stopped

    # Resources
    deploy:
      resources:
        limits:
          cpus: '0.5'
          memory: 512M

    # Logging
    logging:
      driver: json-file
      options:
        max-size: "10m"
        max-file: "3"

    # Other
    container_name: my-container
    command: npm start
    working_dir: /app
    user: "1000:1000"

volumes:
  data:

networks:
  frontend:
```


Services (Услуги)

Всеки service:

- Използва image или се build-ва от Dockerfile
- Може да има множество instances (scaling)
- Получава автоматично DNS име (service name)
- Може да комуникира с други services по име

Service Lifecycle

Lifecycle етапи:

1. **Created** - контейнерът е създаден но не е стартиран
2. **Running** - контейнерът работи и processes са активни
3. **Paused** - контейнерът е временно паузиран (freeze на процесите)
4. **Stopped/Exited** - контейнерът е спрян (може да се рестартира)
5. **Removed** - контейнерът е изтрит (заедно с неговите данни)

Service Lifecycle

```
services:
  api:
    image: node:18-alpine
    restart: unless-stopped      # Автоматичен restart
    depends_on:
      - db                      # Зависимости
    healthcheck:
      test: ["CMD", "curl", "-f", "http://localhost:3000/health"]
      interval: 30s
```

Service Lifecycle

```
docker compose up      # Created → Running
docker compose pause   # Running → Paused
docker compose unpause  # Paused → Running
docker compose stop     # Running → Stopped
docker compose start    # Stopped → Running
docker compose restart  # Stopped → Running (или Running → Stopped → Running)
docker compose down     # Stopped → Removed
```

Service Naming и DNS Resolution

Compose автоматично създава DNS записи:

```
services:
  backend:
    image: myapp:latest

  database:
    image: postgres:15

# Backend може да достъпи database на:
# - database:5432 (service name)
# - postgres:5432 (ако е зададено с aliases)
```

```
services:
  api:
    networks:
      mynetwork:
        aliases:
          - api-service
          - backend-api

# Достъпен на:
# - api:5000
# - api-service:5000
# - backend-api:5000
```


Scaling Services

```
services:
  worker:
    image: myapp-worker:latest
    # 5E3 container_name 3a scaling
    deploy:
      replicas: 3   # 3a Swarm mode
```

Типове сървиси

Категория	Подкатегории	Ключови характеристики	Примери / Use cases
Network Exposure	- Public-facing - Internal - Host Network	- Public: има port mapping, достъпен отвън - Internal: достъпен само вътрешно чрез expose - Host: директен достъп до host network, без изолация	Web server, API gateway, internal microservice, monitoring tools
State Management	- Stateful - Stateless - In-Memory/tmpfs	- Stateful: има volumes - Stateless: без локално състояние - In-memory: временни данни в RAM	Databases, API servers, Redis cache
Lifecycle Management	- Long-running - One-time/Init - On-failure Restart	- Long-running: постоянно активни - One-time: изпълняват се веднъж - On-failure: рестартират се при грешка	Web/API servers, migrations, background jobs
Scaling Capabilities	- Scalable - Single-instance	- Scalable: без container_name, без fixed ports - Single-instance: фиксирани имена/портове	Workers, Databases
Build Strategy	- Pre-built - Built - Hybrid	- Pre-built: готов image - Built: build локално - Hybrid: build + tag за registry	CI/CD pipelines, production builds
Environment-specific	- Production / Development / Testing	- Използване на profiles - Различни настройки за всяка среда	Dev tools, test DBs, staging configs

Роля	Подтипове Services	Основна функция	Примери / Технологии
Web/Frontend Services	- Static Content Server - SPA Application - Reverse Proxy	Сервиране на UI или frontend ресурси	Nginx, React/Vue build, Nginx proxy
API/Backend Services	- REST API - GraphQL Server - Internal Microservices	Обработка на бизнес логика и заявки	Node.js, Express, FastAPI, Django
Database Services	- PostgreSQL - MySQL/MariaDB - MongoDB - Redis (Persistent)	Съхранение на данни	PostgreSQL, MongoDB, Redis
Cache & Message Queue Services	- Redis (Cache Mode) - Memcached - RabbitMQ - Kafka	Кеширане и асинхронна комуникация	Redis, RabbitMQ, Kafka
Background Workers	- Celery Worker - Celery Beat - Job Processor - Cron	Обработка на задачи във фонов режим	Celery, Node Workers, Cron
Utility & Init Services	- Migration - Seeding	Инициализация и поддръжка на приложение	npm migrate, seeders, curl healthcheck
Development Tools	- MailHog - Swagger UI - pgAdmin	Dev помощни услуги	MailHog, Admin UI, Swagger docs

Networks (Мрежу)

Networks позволяват на services да комуникират помежду си и контролират network isolation.

Default Network Behavior

По подразбиране Compose създава bridge network за всички services:

```
# docker-compose.yml
services:
  web:
    image: nginx
  api:
    image: node:18

# Автоматично създава: myproject_default network
# web и api могат да комуникират на web:80 и api:3000
```

Custom Networks

Създавайте custom networks за по-добър контрол:

```
services:
  frontend:
    networks:
      - public

  api:
    networks:
      - public
      - backend

  database:
    networks:
      - backend # Само вътрешна комуникация

networks:
  public:
    driver: bridge
  backend:
    driver: bridge
```

Network Drivers

Bridge (default):

```
networks:
  mynetwork:
    driver: bridge
    ipam:
      config:
        - subnet: 172.28.0.0/16
```

Host (директен достъп до host network):

```
services:
  app:
    network_mode: "host" # Използва host network stack
```

Network Drivers

None (без network):

```
services:  
  isolated:  
    network_mode: "none"
```

Overlay (multi-host, Docker Swarm):

```
networks:  
  myoverlay:  
    driver: overlay  
    attachable: true
```

Задавайте допълнителни DNS имена

```
services:
  api:
    networks:
      frontend:
        aliases:
          - api-server
          - backend-service
      backend:
        aliases:
          - internal-api

networks:
  frontend:
  backend:
```


Inter-Service Communication

```
services:
  nginx:
    image: nginx:alpine
    depends_on:
      - api
    networks:
      - webnet

  api:
    image: node:18
    environment:
      DATABASE_URL: postgresql://db:5432/myapp
    networks:
      - webnet
      - dbnet

  db:
    image: postgres:15
    networks:
      - dbnet

networks:
  webnet:      # Frontend layer
  dbnet:       # Backend layer (изолирана)
  internal: true # Без external connectivity
```

Volumes (Томове)

Volumes осигуряват persistent storage за контейнери и позволяват споделяне на данни.

Named Volumes (препоръчано за production)

```
services:
  db:
    volumes:
      - postgres-data:/var/lib/postgresql/data

volumes:
  postgres-data: # Управлява се от Docker
```

web и api могат да комуникират на web:80 и api:3000

TuneBe Volumes

tmpfs Mounts (temporary, в RAM):

```
services:  
  app:  
    tmpfs:  
      - /tmp  
      - /run
```

Bind Mounts (за development):

```
services:  
  app:  
    volumes:  
      - ./src:/app/src          # Относителен път  
      - /host/path:/app/data    # Абсолютен път
```


Volume Sharing

```
services:
  app:
    volumes:
      - shared-data:/app/data

  backup:
    volumes:
      - shared-data:/backup/source:ro # read-only

volumes:
  shared-data:
```

Read-only volumes:

```
services:
  app:
    volumes:
      - ./config:/app/config:ro
      - data-volume:/app/data:rw # read-write (default)
```

Anonymous Volumes

```
services:
  node-app:
    volumes:
      - ./src:/app/src          # Bind mount
      - /app/node_modules      # Anonymous volume (override)
```

Environment Variables

Environment variables конфигурират services без hardcoding values.

Inline в compose файла:

```
services:
  app:
    environment:
      NODE_ENV: production
      PORT: 3000
      DEBUG: "false"
```

Environment Variables

List формат:

```
services:
  app:
    environment:
      - NODE_ENV=production
      - PORT=3000
```

От .env файл (автоматично):

```
# .env файл
DATABASE_URL=postgresql://localhost/mydb
API_KEY=secret123

# docker-compose.yml
services:
  app:
    environment:
      DATABASE_URL: ${DATABASE_URL}
      API_KEY: ${API_KEY}
```

Environment Variables

От custom файл:

```
services:
  app:
    env_file:
      - .env
      - .env.production
      - secrets.env
```

Variable Substitution

```
services:
  app:
    image: ${APP_IMAGE:-myapp:latest} # Default value
    environment:
      API_URL: ${API_URL}
      TIMEOUT: ${TIMEOUT:-30}
      DEBUG: ${DEBUG:-false}
```


Dependencies (Зависимости)

Dependencies контролират реда на стартиране на services.

```
services:  
  app:  
    depends_on:  
      - db  
      - redis  
  
  db:  
    image: postgres:15  
  
  redis:  
    image: redis:7
```

Dependencies (Зависимости)

Health Check Conditions

```
services:
  app:
    depends_on:
      db:
        condition: service_healthy
        restart: true
      redis:
        condition: service_started

  db:
    image: postgres:15
    healthcheck:
      test: ["CMD-SHELL", "pg_isready"]
      interval: 10s
      timeout: 5s
      retries: 5

  redis:
    image: redis:7
```

- service_started - контейнерът е стартиран (default)
- service_healthy - health check е successful

YAML Файл Структура

```
version: '3.8' # Опционално в нови версии

services:
  service-name:
    # Build или Image
    image: image:tag
    # или
    build:
      context: ./path
      dockerfile: Dockerfile

    # Ports
    ports:
      - "host:container"

    # Environment
    environment:
      KEY: value

    # Volumes
    volumes:
      - host-path:container-path

    # Networks
    networks:
      - network-name

    # Dependencies
    depends_on:
      - other-service

    # Restart policy
    restart: unless-stopped

    # Command override
    command: ["custom", "command"]

networks:
  network-name:
    driver: bridge

volumes:
  volume-name:
    driver: local
```


Версии на Compose файл формата

- Version 1 (legacy) - оригинален формат
- Version 2 - добавя networks, depends_on
- Version 3 - за Docker Swarm compatibility
- Latest (no version) - модерен формат без version field